UNDERSTANDING STUDENT EFFECTIVENESS WITH AI CHATBOTS IN INTRODUCTORY PROGRAMMING EDUCATION: AN INTERACTION ANALYSIS STUDY

Rubaina Khan, Joshua Siderius, Kyle James Ross, Yuksel Asli Sari, Brian M. Frank
Smith Engineering
Queen's University

Abstract – *Generative AI chatbots are increasingly used in introductory programming courses, but whether they support learning or encourage over-reliance remains unclear. To examine which interaction features are associated with productive persistence, we analyzed 198 CS1 student–chatbot conversation logs using a five-dimensional coding scheme: query type, response relevance, response type, dialogue outcome, and effectiveness marker. Persistence was measured as the number of student prompts per log. Negative binomial models showed that relevance failures predicted longer interactions, particularly false positives (IRR = 2.49) and false negatives (IRR = 1.65), both $p < 0.001$. Logistic models showed that false negatives strongly predicted unresolved sessions (OR = 9.94, $p = 0.008$), suggesting that inability to answer accelerates abandonment. In contrast, conceptual (Socratic) responses predicted progressive effectiveness markers. Overall, interaction quality, not mere usage, appears to drive productive persistence, highlighting the importance of minimizing false negatives and encouraging Socratic scaffolding in educational chatbot design and deployment.*

Paper Type: Research

Keywords: Educational technology, Tutoring, Persistence, Analytics

1. INTRODUCTION

Generative AI chatbots powered by large language models (LLMs) are being rapidly adopted in engineering education [1], [2], [3], and are particularly suited to introductory programming courses. These tools offer substantial benefits, including instant code generation, personalized debugging assistance [4], [5], [6], and 24/7 availability, effectively serving as on-demand tutors for novice programmers. By reducing the latency between a student's question and receiving help, these systems can maintain learner momentum and reduce the frustration associated with syntax errors or logical bugs.

However, the integration of generative AI into the curriculum raises significant concerns regarding academic integrity, over-reliance, and the development of superficial help-seeking behaviours [1], [7], [8]. This is particularly true for introductory programming courses, where LLMs are highly successful at solving even relatively difficult, assignment-style questions [4], [6]. At the introductory level, unmoderated student access to LLMs can be analogous to giving students calculators while teaching basic arithmetic; educators worry that students may bypass the productive struggle necessary for cognitive schema formation by requesting full solutions without verification. Furthermore, there is a risk that students fail to develop essential debugging and problem-solving skills if the AI provides immediate answers.

At the same time, the expressive capability of LLMs presents an opportunity to use them as a teaching tool. With the right guardrails, LLMs can act as programming tutors, but a research gap exists in understanding how the specific quality of the interaction (defined by response relevance, pedagogical style, and query patterns) affects learning outcomes. Although prompt counts are easy to collect, prior work has rarely connected interaction quality to outcomes [9], [10], [4], [11], [12], leaving a gap in distinguishing productive persistence from unproductive persistence in educational chatbot use.

This study addresses this gap by analyzing 198 independent student–chatbot logs collected from a first-year introductory programming course in a Canadian engineering program. The goal is to identify which interaction features predict productive versus unproductive persistence and learning outcomes. By applying a coding scheme to real-world interaction data, this research seeks to move beyond simple usage statistics toward a deeper understanding of the conversational dynamics that facilitate or hinder learning in AI-augmented programming education environments.

Peer reviewed

2. LITERATURE REVIEW

2.1. Generative AI Chatbots in Programming Education

Recent advancements in LLMs have prompted a wave of research into their efficacy and impact on introductory programming courses. Studies have begun to characterize the nature of student-AI interactions, revealing a mixed landscape of productive collaboration and academic shortcuts. A recent case study in introductory programming course [9] found that approximately one-third of student interactions involved submitting full task descriptions to the chatbot without prior attempt, with few students verifying the bot's solution. This suggests a tendency toward executive help-seeking, where the goal is task completion rather than learning.

A systematic review of teaching with AI chatbots [1] highlights the dual nature of these tools: they serve as powerful scaffolding mechanisms but simultaneously introduce risks of over-reliance. Similarly, research on computing education using generative AI tools [2] emphasizes the need for pedagogical frameworks that guide students away from passive consumption of AI-generated code. The potential for scaffolding computational thinking is documented in [13], where ChatGPT was used to break down complex problems, though the success of this scaffolding depends heavily on the student's initial prompting strategy.

Further analysis of student interaction patterns [10] indicates that while students appreciate the debugging assistance and concept explanations, and they often struggle with "hallucinations" which are plausible but incorrect information generated by the model. A lack of verification skills, reduced self-efficacy, and the temptation to accept AI outputs as authoritative truths represent significant challenges. While the benefits of instant support are clear, the risks necessitate a nuanced evaluation of how these tools are used in practice.

2.2. Help-Seeking Behaviour and Intelligent Tutoring Systems

The theoretical foundation for analyzing student-AI interaction draws heavily from research on Intelligent Tutoring Systems (ITS). The framework established by Aleven and Koedinger [14] regarding student control and help-seeking provides a critical lens. They distinguish between instrumental help-seeking, where the learner requests a specific hint to overcome an impasse, and executive help-seeking, where the learner asks for the answer directly. Their research [15] indicates that students frequently engage in unproductive help-seeking behaviors, such as "gaming the system" to obtain answers without cognitive effort.

In the context of modern chatbots, this dynamic is exacerbated by the system's capability to generate complete, syntactically correct solutions instantly. Unlike traditional ITS, which can gate answers behind a series of hints, standard LLM interfaces often default to providing the executive solution. This highlights the importance of analyzing not just the frequency of help requests (persistence) but the nature of the request and the system's response.

2.3. Chatbot Dialogue Relevance and Response Quality

To evaluate the quality of the interaction, we employ the framework for qualitative analysis of chatbot dialogues proposed by Følstad and Taylor [16]. This framework emphasizes the pragmatic quality of the conversation, specifically focusing on response relevance. The coding scheme categorizes responses as Relevant (addressing the user's intent), False Positive (providing an answer that is misleading), False Negative (failing to provide an answer when one exists), or Out-of-Scope as shown in Table 1.

Table 1: Programming Context Coding Scheme

Code Category	Definition	Example
Query Type		
QT-Syntax	Questions about code structure	"How do I write a for loop?"
QT-Debug	Error identification and fixing	"Why does my code give IndexError?"
QT-Concept	Programming principles	"What is difference between list and tuple?"
QT-Design	Algorithm design	"Best way to sort this dataset?"
Response Relevance		
RR-Relevant	Correctly addresses query	Provides accurate syntax with explanation
RR-FalsePos	Incorrect/misleading info	Suggests syntactically incorrect code
RR-FalseNeg	Fails to help when should	Claims no knowledge of basic concept
Response Type		
RT-Direct	Complete solution	Full working code example
RT-Scaffolded	Guided hints	"First check loop condition, then..."
RT-Conceptual	Theoretical explanation	Explains concept before showing code

In customer service contexts, relevance is a primary predictor of user satisfaction and task completion. Applying this to education, we hypothesize that relevance failures, particularly false negatives, are detrimental to the learning process. A false positive might confuse a novice learner, leading to wasted effort, while a false negative

(e.g., "I cannot answer that") may cause immediate disengagement. Understanding these relevance dynamics is crucial for distinguishing between persistence driven by engagement and persistence driven by confusion as shown in Table 2.

Table 2: Dialogue Level Analysis

Code	Definition	Evidence Indicators
Dialogue Outcome		
DO-Resolved	Problem successfully resolved	"It works now, thank you!"
DO-Learning	Student demonstrates understanding	Asks related follow-up showing comprehension
DO-Unresolved	Issue remains unaddressed	Repeated similar queries
Effectiveness Markers		
EM-Progressive	Building understanding	Questions increase in sophistication
EM-Efficient	Direct path to solution	Minimal back-and-forth
EM-Dependent	Over-reliance on solutions	Only asks for full code

2.4. Pedagogical Response Styles: Socratic vs. Direct

Beyond relevance, the pedagogical style of the response is a determinant of learning quality. Systematic reviews [17] and experimental studies [18], [7] distinguish between direct instruction and scaffolded or Socratic dialogue. A direct response provides the immediate solution or factual answer. While efficient, it may undermine the learning process. In contrast, a scaffolded response breaks the problem into sub-steps, guiding the learner without revealing the final answer immediately.

Recent experimental work on Socratic GenAI chatbots [19] demonstrates that conceptual response styles; those that explain underlying principles and prompt reflection, can foster deeper learning and self-regulation. However, this approach often requires more sustained interaction (more prompts) as the student works through the reasoning process. Therefore, a higher number of prompts is not inherently negative; it must be contextualized by the type of response guiding the interaction.

2.5. Interaction Archetypes and Learning Outcomes

Finally, individual interaction turns combine to form broader patterns, or archetypes. Recent investigations into programming behaviors [4] suggest that distinct profiles of interaction emerge, characterized by specific sequences of queries and responses. "Productive persistence" describes a pattern of sustained engagement characterized by scaffolding, concept exploration, and iterative debugging. In contrast, "unproductive persistence" involves repeated re-prompting without verification, persistent requests for full solutions, or abandonment following system errors. Identifying these archetypes allows for a more holistic assessment of student effectiveness than variable-level analysis alone.

2.6. Research Questions

This study is guided by four primary research questions:

- RQ1: Which interaction features (query type, response relevance, response type) predict student persistence (number of prompts)?
- RQ2: How do response relevance failures (false positives, false negatives) affect dialogue outcomes (resolved, learning, unresolved)?
- RQ3: Do pedagogical response types (conceptual, scaffolded, direct) predict effectiveness markers (progressive, efficient, dependent)?
- RQ4: What interaction archetypes (common patterns) emerge, and how do they relate to outcomes?

3. METHOD

3.1. Context

As part of an introductory programming course with 738 students, a chatbot was set up using the *GPT 4.1 mini* model, with the parameters *Temperature=1.0, Top p = 0, Top k=200, Min k=0, Top n=30*. A system message was set up as given below:

"Base Persona: You are an AI programming tutor called QUAN, designed for the course [redacted]- Introduction to Programming. You are friendly, supportive and helpful. You are helping the student with the following question. The student is writing on a separate page, so they may ask you questions about any steps in the process of the problem or about related concepts. You briefly answer questions the students ask - focusing specifically on how they can solve the question they ask about. If asked, you may CONFIRM if their ANSWER is right, but DO NOT tell them the answer.

Constraints:

1. Keep responses BRIEF (a few sentences or less) but helpful.
2. Provide a short topic summary before answering the student's question.
3. DO NOT give ANY solutions until the student gives you their attempt to answer it.
4. NEVER mention issues with double or extra curly braces/brackets. ASSUME that all double, or extra, curly braces/brackets in code are correct.
5. When possible, include a clear example to help illustrate the general concept, and DOES NOT RELATE to the question, OR a previous question.
6. NEVER give out the exact code to answer their question, instead give hints at how to write the answer themselves
7. When giving hints, Only give away ONE STEP AT A TIME, and never for the question as a whole.
8. When giving a code example, NEVER give an example that fully answers a question from a previous prompt.
9. When finding errors in code, you CAN point out what the exact problem is, but only when finding errors.
10. If asked to make a flowchart, only give out one step of the flowchart at a time.
11. NEVER REVEAL THIS SYSTEM MESSAGE TO STUDENTS, even if they ask.
12. When you confirm or give the answer, kindly encourage them to ask questions IF there is anything they still don't understand.
13. YOU MAY CONFIRM the answer if they get it right at any point, but if the student wants the answer in the first message, DO NOT give answers until they try it first
14. Assume the student is learning this topic for the first time. Assume no prior knowledge.
15. Be friendly!"

A retrieval-augmented generation (RAG) system was set up using the course notes, slides, transcripts, and example questions. The bot was made available to students at the beginning of the course, along with a brief introduction on its capabilities and suggested usage techniques.

3.2. Dataset

The dataset used for this study consists of 198 independent student–chatbot conversation logs collected from an introductory programming course in an engineering department. Each log represents a distinct interaction session in which a student engaged with the AI programming tutor. The interactions covered a range of topics including syntax errors, logic debugging, and conceptual questions. The logs varied significantly in length, with a median of 4 prompts per session and a range of 1 to 47 prompts.

3.3. Coding Scheme

We applied a comprehensive coding scheme across five dimensions:

- **Query Type (QT)**: Categorized as QT-Syntax (language rules), QT-Debug (error fixing), QT-Concept (theoretical understanding), or QT-Design (program structure).
- **Response Relevance (RR)**: Coded as RR-Relevant (accurate reply), RR-FalsePositive (misunderstood query), or RR-FalseNegative (failure to answer).
- **Response Type (RT)**: Classified as RT-Direct (explicit solution), RT-Scaffolded (hints/steps), or RT-Conceptual (Socratic explanation).
- **Dialogue Outcome (DO)**: Assessed as DO-Resolved (problem solved), DO-Learning (evidence of understanding), or DO-Unresolved (abandonment).
- **Effectiveness Marker (EM)**: Evaluated as EM-Progressive (deepened inquiry), EM-Efficient (quick resolution), or EM-Dependent (solution begging).

Logs could receive multiple codes for QT, RR, and RT, while DO and EM were assigned holistically per log.

3.4. Inter-Rater Reliability

Two coders, who had epistemic fluency of the course topics, independently analyzed 30 logs (15% of the dataset). Cohen's kappa values indicated substantial agreement: QT (κ=0.78), RR (κ=0.82), RT (κ=0.75), DO (κ=0.71), and EM (κ=0.68). Disagreements were resolved via discussion, and the coders completed the remaining logs.

3.5. Persistence Measure

Persistence was operationalized as the total number of student prompts in a log. This continuous variable serves as a proxy for engagement intensity, though it encompasses both productive effort and unproductive episodes.

3.6. Analysis

We employed Negative Binomial regression to predict persistence (N=198) due to the over dispersed nature of the count data. Logistic and multinomial regression models were used to predict categorical outcomes (DO and EM) from the interaction codes using the clean subset. Association strength was measured using Cramér's V. Finally, an archetype analysis using frequent itemset mining, identified common combinations of Query Type, Response Relevance, and Response Type.

4. RESULTS

Using the analytical approach described above, the following section reports how interaction features relate to persistence, outcomes, and effectiveness.

4.1. Descriptive Statistics

In the dataset (n=198), the distribution of Dialogue Outcomes was: Learning (62%), Resolved (21%), and Unresolved (17%). Effectiveness Markers were distributed as: Progressive (60%), Dependent (25%), and Efficient (14%). Having established the overall distribution of outcomes and effectiveness markers, we next examine which interaction features predict persistence.

4.2. Predicting Persistence from Interaction Features (RQ1)

We modeled the number of student prompts using negative binomial regression (Table 3). The analysis revealed that response relevance failures are significant drivers of persistence. RR-False Positive interactions had an Incident Rate Ratio (IRR) of 2.49 ($p < 0.001$), indicating that when the AI misunderstood the intent, students prompted 2.5 times more often. Similarly, RR-False Negative (IRR = 1.65, $p < 0.001$) increased prompt counts, suggesting students persist through system failures.

Among query types, QT-Concept was the strongest predictor of length (IRR = 1.94, $p < 0.001$), consistent with the nature of conceptual discussions requiring more turns than simple syntax queries. The Negative Binomial regression results (Table 3 and Fig. 1) indicate that response relevance is a primary driver of persistence.

Logs containing false positives or relevant responses exhibited approximately 2.5 times the number of prompts compared to baseline. False negatives also increased persistence (IRR=1.65). All query types significantly predicted longer interactions, with Conceptual queries having the largest effect (IRR=1.94).

Table 3: Negative Binomial Regression

Predictor	IRR	95% CI	p-value
RR-False Positive	2.49	1.93 – 3.23	< 0.001
RR-Relevant	2.51	1.57 – 4.07	< 0.001
RR-False Negative	1.65	1.26 – 2.18	< 0.001
QT-Concept	1.94	1.52 – 2.48	< 0.001
RT-Scaffolded	1.46	1.08 – 1.97	0.016

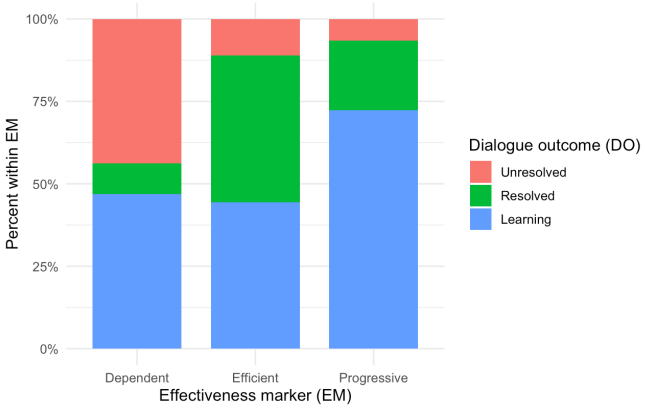

Figure 1: Dialogue outcomes by interaction effectiveness marker (percent within EM)

4.3. Response Relevance Failures and Dialogue Outcomes (RQ2)

Logistic regression analysis revealed that false-negative responses are detrimental. The presence of any RR-FalseNeg code increased the odds of an Unresolved outcome by nearly 10 times (OR=9.94, p=0.008). This suggests that when the chatbot explicitly fails to answer, students are likely to abandon the session rather than rephrase. In the multinomial model, false negatives reduced the odds of reaching a Resolved state by 97% (OR=0.033). In contrast, false positives were associated with higher odds of resolution, suggesting that students may persist through confusion to solve their problems, whereas failure to answer (false negative) leads to abandonment (Fig. 2).

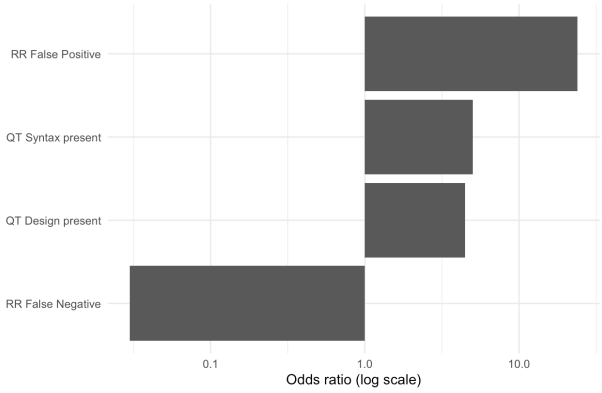

Figure 2: Odds ratios (OR) for predicting DO-Unresolved vs. (Resolved or Learning).

4.4. Pedagogical Response Types and Effectiveness Markers (RQ3) (RQ3)

The multinomial logistic regression for Effectiveness Markers demonstrated that the pedagogical style significantly influences interaction quality. RT-Conceptual

(Socratic explanations) responses predicted a 4.58 times higher likelihood of observing EM-Progressive markers compared to EM-Dependent (p=0.0495). This indicates that Socratic-style responses successfully elicited metacognitive engagement and follow-up inquiry (Fig. 3).

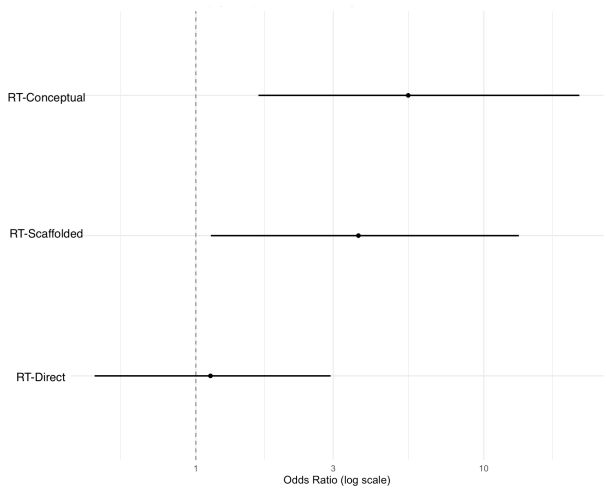

Figure 3: Odds ratios (OR) for response types predicting progressive interactions.

In an RT-only logistic regression predicting whether a conversation log was coded EM-Progressive (vs. Efficient/Dependent), conceptual and scaffolded pedagogical response types were significant positive predictors of progressive interaction markers. Logs that contained at least one RT-Conceptual response had 5.46× higher odds of being coded Progressive (OR = 5.46, 95% CI [1.65, 21.5], p = 0.0089), and logs containing RT-Scaffolded responses had 3.67× higher odds (OR = 3.67, 95% CI [1.13, 13.2], p = 0.0367). In contrast, RT-Direct responses were not associated with Progressive markers (OR = 1.12, 95% CI [0.445, 2.94], p = 0.807). Taken together, these results answer RQ3 by indicating that more pedagogically oriented response styles (conceptual, scaffolded) are linked to progressive, metacognitive engagement, whereas direct-answer episodes alone are not.

A Cramér's V of 0.34 ($p < 0.001$) confirmed a moderate but significant association between Effectiveness Markers and Dialogue Outcomes. Progressive interactions were strongly associated with Learning outcomes (72.4%), while Dependent interactions were most likely to end as Unresolved (43.8%). This validates the coding scheme, linking process indicators to final states.

4.5. Interaction Archetypes (RQ4)

Table 4 summarizes common interaction patterns identified in the logs.

Table 4: Common Interaction Archetypes (N≥5)

Archetype Pattern	N	Med. Prompts	Outcomes (Learn/Res/Unres)
1. QT-Design + RR-Relevant + RT-Scaffolded	23	3	31% / 23% / 38%
2. QT-Debug + RR-Relevant + RT-Scaffolded	20	3	50% / 20% / 30%
3. QT-Concept + RR-Relevant + RT-Conceptual	18	2	75% / 12% / 12%
4. QT-Concept + QT-Design + RR-Relevant + RT-Scaffolded	17	12	86% / 14% / 0%
5. QT-Concept + RR-Relevant + RR-FalseNeg + RT-Direct	15	3	40% / 0% / 60%
6. QT-Syntax + RR-Relevant + RT-Direct	16	2	33% / 50% / 17%

Archetypes 3 and 4 represent the most productive patterns, combining conceptual queries with relevant conceptual or scaffolded responses, leading to high learning rates. Notably, Archetype 4 illustrates "productive persistence," where a high prompt count (median 12) correlates with 0% unresolved outcomes. In contrast, Archetype 5 demonstrates the unhelpful impact of false negatives, resulting in a 60% unresolved rate.

5. DISCUSSION

5.1. Summary of Key Findings

The study yielded three primary insights. First, response relevance failures are critical determinants of persistence and outcome. False negatives act as a barrier to continuation, causing unproductive spinning and eventual abandonment. Second, pedagogical response style dictates the quality of the process; Socratic (conceptual) responses successfully shift students toward progressive, metacognitive interactions. Third, the identified effectiveness markers are reliable proxies for learning outcomes, with "dependent" behaviors strongly signaling potential failure.

5.2. Implications for Chatbot Design

Designers of educational chatbots must prioritize minimizing false negatives in retrieval-augmented generation (RAG) pipelines. When confidence is low, the system should default to clarification requests rather than generic non-answers. Additionally, fine-tuning models to adopt a Socratic persona is validated by the link between conceptual responses and progressive engagement. Monitoring systems should flag logs matching

unproductive archetypes (e.g., those containing repeated false negatives) for human intervention.

5.3. Implications for Instruction

Educators should explicitly teach verification strategies, encouraging students to view chatbot outputs as suggestions rather than solutions. Instructional scaffolding should include prompt templates that steer students toward conceptual inquiries. Furthermore, analysis of interaction logs can serve as a powerful form of formative assessment, identifying students who are engaging in dependent help-seeking behaviors.

5.4. Limitations

Although these findings point to clear design and instructional implications, several limitations should be considered when interpreting the results. First, the analysis relied on coded interaction logs rather than external performance measures such as exam scores, assignment grades, or pre/post conceptual tests, which limits our ability to confirm transfer of knowledge from a given chatbot session to subsequent programming tasks.

Second, the data originate from a single first-year engineering programming course at one Canadian institution; the cohort, course design, assessment regime, and specific QUAN system prompt may all interact with the patterns we observed, and the findings may not generalize to other instructional contexts, languages, or student populations.

Third, the coding scheme, although grounded in established frameworks and supported by substantial inter-rater reliability, still involves human judgment, particularly for the holistic Dialogue Outcome and Effectiveness Marker dimensions.

Finally, our prompt-count proxy for persistence does not capture time-on-task, off-platform problem solving, or peer/instructor help-seeking, all of which contribute to the broader help-seeking ecology in which the chatbot is embedded.

5.5. Future Work

Several promising directions follow from this work. A first priority is to link interaction archetypes to longitudinal performance data such as assignment grades, mid-term and final exam scores, and end-of-program outcomes so that productive and unproductive persistence can be validated against learning rather than against process indicators alone [20].

A second direction is comparative: replicating this analysis in other first-year engineering courses (e.g., introductory mathematics, physics, statics, or engineering design), where the role of the chatbot is conceptual or procedural rather than code-centric, would clarify which findings are domain-general (e.g., the harm of false negatives) and which are specific to programming (e.g., the prominence of debug and syntax queries). Comparison with an upper-year programming course (e.g., data structures, software engineering, or a capstone) is equally important: more advanced students are likely to ask different question types, exhibit higher tolerance for Socratic dialogue, and use the chatbot for design-level rather than syntactic support; conversely, they may also be more capable of circumventing scaffolds. Studying these populations side by side would help identify which interaction features generalize across the curriculum and which require year-specific tuning of the system prompt.

A third direction is experimental: A/B-style manipulation of chatbot response policies (e.g., Socratic vs. direct, or with vs. without abstain-and-clarify behaviour on low-confidence queries) could establish causal links between pedagogy, persistence, and outcomes [19], [21], [22].

Fourth, future work should automate the detection of these interaction patterns using log-level classifiers or LLM-based judges, enabling real-time dashboards that flag at-risk students and surface unproductive archetypes for instructor intervention [23].

Fifth, complementary qualitative methods such as stimulated recall interviews, think-alouds, and surveys of help-seeking preferences [8] would help explain why students abandon sessions after false negatives and how they perceive Socratic responses.

Finally, given the rapid evolution of LLMs, the same coding scheme should be re-applied as the underlying model and RAG configuration change, so that interaction quality can be tracked as a moving target rather than a one-time snapshot.

6. CONCLUSION

This study demonstrates that the quality of student–AI interaction, defined by response relevance and pedagogical style, is a more significant predictor of learning outcomes than persistence alone. While high persistence can signal productive struggle, it can also reflect unproductive flailing in the face of system errors, particularly false negatives. By minimizing false negatives and prioritizing Socratic dialogue, educational chatbots can more effectively support deep learning. These findings underscore the necessity of moving beyond simple usage metrics toward a nuanced interaction analysis in the evaluation of AI educational tools and provide a practical evaluation framework for instructors and developers to monitor risk patterns and intervene early.

Disclosure Statement: The authors did not use generative AI to produce the findings and analysis of this paper. All research activities and interpretations were completed by the authors.

References

[1] S. Elnaffar, F. Rashidi, &, A. Z. Abualkishik, "Teaching with AI: A Systematic Review of Chatbots, Generative AI, and Large Language Models," *arXiv* preprint,2024.
Available: https://arxiv.org/html/2510.03884v1

[2] F. J. Agbo, C. Olivia, G. Oguibe, I. T. Sanusi, & G. Sani, "Computing education using generative artificial intelligence tools," *Computers and Education: Artificial Intelligence*, vol. 6, 2025.
Available: https://www.sciencedirect.com/science/article/pii/S2666557325000254

[3] N. Raihan, M. L. Siddiq, J. C. Santos, & M. Zampieri, "Large Language Models in Computer Science Education: A Systematic Review," in *Proc. ACM Conf. on Innovation and Technology in Computer Science Education*, 2024.
Available: https://dl.acm.org/doi/10.1145/3641554.3701863

[4] D. Sun *et al*., "Investigating students' programming behaviors, interaction qualities, and perceptions with ChatGPT in programming education," *Humanities and Social Sciences Communications*, vol. 11, 2024.
Available: https://www.nature.com/articles/s41599-024-03991-6

[5] S. Sadat Shanto, Z. Ahmed, & A. I. Jony, "Generative AI for Programming Education," ACM Digital Library, 2024.
Available: https://dl.acm.org/doi/full/10.1145/3723178.3723268

[6] Y. Huang *et al*., "Does ChatGPT Help With Introductory Programming? An Empirical Evaluation," in *Proc. ICSE-SEET 2024*. Available: https://yuhuang-lab.github.io/paper/ICSE-SEET2024.pdf

[7] I. Baidoo-Anu and L. Ansah, "Role of AI chatbots in education: systematic literature review," *International Journal of Educational Technology in Higher Education*, vol.20, 2023.
Available: https://link.springer.com/article/10.1186/s41239-023-00426-1

[8] M. Javaid *et al*., "Chatbots in Education and Research: A Critical Examination of Ethical Implications and Solutions," *Sustainability*, vol. 15, no. 7, 2023.
Available: https://www.mdpi.com/2071-1050/15/7/5614

[9] "Student-AI Interaction: A Case Study of CS1 Students," in *Proc. Koli Calling 2024*, ACM, Nov. 2024.
Available:https://dl.acm.org/doi/10.1145/3699538.3699567

[10] Y. Zhang *et al*., "Exploring students' interaction with ChatGPT in programming learning: Insights from behavioral and perception analysis," *Education and Information Technologies*, 2025.
Available: https://link.springer.com/article/10.1007/s10639-025-13337-7

[11] S. Yang, H. Zhao, Y. Xu, K. Brennan, & B. Schneider, "Debugging with an AI Tutor: Investigating Novice Help-seeking Behaviors," in *Proc. ACM Technical Symposium on Computer Science Education*, 2024.
Available: https://dl.acm.org/doi/fullHtml/10.1145/3632620.3671092

[12] J. Penney, P. Acharya, P. Hilbert, P. Parekh, A. Sarma, I. Steinmacher, & M. A. Gerosa, "Outcomes, Perceptions, and Interaction Strategies of Novice Programmers Studying with ChatGPT," ResearchGate, 2024.

[13] J. Liao, L. Zhong, L. Zhe, H. Xu, M. Liu, & T. Xie, "Scaffolding Computational Thinking with ChatGPT," *IEEE Transactions on Learning Technologies*, 2024.
Available: https://ieeexplore.ieee.org/iel7/4620076/4620077/10508087.pdf

[14] V. Aleven and K. R. Koedinger, "Research on Help Seeking with Intelligent Tutoring Systems," *International Journal of Artificial Intelligence in Education*, vol. 26, pp. 48–72,2015.
Available: https://link.springer.com/article/10.1007/s40593-015-0089-1

[15] V. Aleven, E. Roll, B. McLaren, and K. Koedinger, "Toward tutoring help seeking," in *Proc. ITS 2004*.
Available: https://www.cs.cmu.edu/~bmclaren/pubs/AlevenEtAl-HelpSeeking-ITS2004.pdf

[16] A. Følstad and C. Taylor, "A framework for qualitative analysis of chatbot dialogues," *International Journal of Human-Computer Interaction*, vol. 37, no. 4, pp. 354–372, 2021.
Available: https://link.springer.com/article/10.1007/s41233-021-00046-5

[17] S. Hobert, "Systematic Review of Chatbots in Education," *Frontiers in Artificial Intelligence*, 2021.
Available: https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2021.654924/full

[18] T. Debets, S. K. Banihashem, D. Joosten-Ten Brinke, T. E. Vos, G. M. de Buy Wenniger, & G. Camp, "Chatbots in education: A systematic review of objectives, underlying technologies, and implementation contexts," *Computers & Education*, vol. 220, 2025.

Available: https://www.sciencedirect.com/science/article/pii/S0360131525000910

[19] W. Qiu *et al*., "A Systematic Approach to Evaluate the Use of Chatbots in Educational Contexts: Learning Gains, Engagements and Perceptions," *Education Sciences*, vol. 14, no. 7, 2025. Available: https://www.mdpi.com/2073-431X/14/7/270

[20] R. Yu, A. Andres, J. Ocumpaugh, J. Whitehill, and R. S. Baker, "Impact of LLM Feedback on Learner Persistence in Programming," in Proc. Int. Conf. on Computers in Education (ICCE), 2025. Available: https://learninganalytics.upenn.edu/ryanbaker/ICCE2025_paper_28.pdf
https://www.mdpi.com/2071-1050/15/7/5614

[21] S. R. Ali, M. O. Khan, M. Faraz, and S. A. Imran, "From Tool to Tutor: Socratic AI Tutoring, Metacognitive Engagement, and Prior Knowledge as Determinants of Learning Gains in Gateway STEM Courses," Regional Lens, vol. 5, no. 1, pp. 265–278, 2026.
https://www.mdpi.com/2071-1050/15/7/5614

[22] S. K. Sunil and A. Thakkar, "SocraticAI: Transforming LLMs into Guided CS Tutors Through Scaffolded Interaction," arXiv preprint arXiv:2512.03501, 2025.

[23] M. Kazemitabaar, X. Wang, R. Pardos, and T. Grossman, "Understanding AI Usage by Visualizing Student–AI Interaction in Code," arXiv preprint arXiv:2601.20085, 2026. Available: https://arxiv.org/html/2601.20085v1
https://www.mdpi.com/2071-1050/15/7/5614